

Plataforma de Telemedicina

Microservicio de Agendamiento de Citas Médicas

Documentación Técnica de la API REST

Información del Proyecto

Componente	Detalle
Metodología de Trabajo	Kanban (Flujo Continuo)
Tecnologías Principales	Python / Flask / SQLAlchemy / PostgreSQL / SQLite
Autores	Darwin García / Juan José Figueroa Ayala
Versión de la API	v1 (Prefijo base: /api/v1)

1. Descripción General

Este microservicio forma parte integral de la *Plataforma de Telemedicina y Triage Automatizado*. Su responsabilidad principal es gestionar el ciclo de vida completo de las citas médicas: desde la creación e inicialización del registro hasta su finalización o cancelación. Adicionalmente, el componente resuelve la consulta de disponibilidad de la agenda del personal médico y la integración de variables con el módulo de triaje automático.

La API se desarrolla bajo el estilo arquitectónico REST, exponiendo recursos mediante semántica HTTP estándar y empleando JSON como formato para el intercambio de datos. El acceso a operaciones sensibles se encuentra restringido y protegido mediante mecanismos de autenticación basados en JWT (JSON Web Tokens).

1.1 Stack Tecnológico

Componente	Tecnología Seleccionada
Lenguaje de Programación	Python 3.x
Framework Web	Flask (Patrón de Fábrica con Blueprints)

Componente	Tecnología Seleccionada
Mapeador Capa Relacional (ORM)	SQLAlchemy + Flask-Migrate (Alembic)
Serialización de Datos	Marshmallow (Flask-Marshmallow)
Mecanismo de Autenticación	JWT mediante Flask-JWT-Extended (Soporte en Header y Cookie)
Sistemas de Gestión de Base de Datos	PostgreSQL (Producción) / SQLite (Entorno de Desarrollo)
Contenedores y Virtualización	Docker + Docker Compose
Política de CORS	Flask-CORS con soporte explícito para credenciales

2. Arquitectura del Microservicio

El diseño de software implementa una arquitectura en capas, aislando las responsabilidades operativas en los siguientes directorios:

Capa / Módulo	Ubicación en Directorio	Responsabilidad Técnica
Rutas (Routes)	app/routes/	Definición de endpoints, validación de peticiones HTTP y delegación a la capa de servicios.
Servicios (Services)	app/services/	Encapsulamiento de la lógica de negocio pura: resolución de conflictos de horario, cálculo de slots y transiciones de estado.
Modelos (Models)	app/models/	Definición de entidades del ORM mapeadas directamente a las tablas relacionales.

Capa / Módulo	Ubicación en Directorio	Responsabilidad Técnica
Esquemas (Schemas)	app/schemas/	Capa de validación de datos de entrada y serialización de estructuras de salida vía Marshmallow.
Utilidades (Utils)	app/utils/	Funciones transversales: estandarización de respuestas, algoritmos de paginación y ayudantes de autenticación.
Configuración	config/config.py	Definición de variables por entorno: desarrollo, pruebas (testing) y producción.
Migraciones	migrations/	Scripts de control de versiones del esquema de datos gestionados por Alembic.

2.1 Flujo de Ejecución de una Petición

1. El cliente emite una petición HTTP incorporando el token JWT en la cabecera Authorization o en la cookie access_token_cookie.
2. Los decoradores de seguridad (@require_jwt o @require_role) interceptan la solicitud y validan la autenticidad y vigencia del token.
3. El controlador (*handler*) en la capa de rutas recibe la petición, valida la estructura del cuerpo (JSON) mediante Marshmallow y delega la ejecución a la capa de servicios.
4. La capa de servicios procesa las reglas de negocio (búsqueda de solapamientos, verificación de agenda) interactuando con la base de datos a través del ORM.
5. El resultado es serializado por los esquemas correspondientes y se retorna envuelto en estructuras homogéneas de respuesta (success_response o error_response).

3. Modelos de Datos

El esquema relacional del microservicio se compone de siete entidades principales encargadas de persistir la información del ecosistema de agendamiento.

3.1 Especialidad

Catálogo que define las ramas de la medicina disponibles en la plataforma. Funciona como nodo raíz para la clasificación del personal médico.

Campo	Tipo de Datos	Restricciones / Descripción
id	Integer	Clave Primaria (PK), autoincremental.
nombre	String(100)	Nombre de la especialidad. Restricción de unicidad (Unique).
descripcion	Text	Detalle de las competencias de la especialidad (Opcional).
activa	Boolean	Flag para borrado lógico (Soft-delete); deshabilita el uso de la especialidad.

3.2 Medico

Representación del personal de salud habilitado para la atención. Mantiene una relación de dependencia con una especialidad específica.

Campo	Tipo de Datos	Restricciones / Descripción
numero_registro	String(30)	Registro médico legal. Clave de negocio con restricción de unicidad (UK).
especialidad_id	Integer	Clave Foránea (FK) referenciando a la entidad Especialidad.
duracion_consulta_min	Integer	Bloque de tiempo estándar en minutos asignado para el cálculo de disponibilidad.
tarifa_consulta	Numeric(10,2)	Costo monetario de la atención médica (Opcional).
activo	Boolean	Estado operativo. Solo los registros activos se incluyen en consultas públicas.

3.3 DisponibilidadMedico

Define la matriz o bloques horarios semanales configurados por cada médico para la provisión de servicios.

Campo	Tipo de Datos	Restricciones / Descripción
medico_id	Integer	Clave Foránea (FK) referenciando a la entidad Medico.
dia_semana	Enum	Valores restringidos: LUNES a DOMINGO.
hora_inicio	Time	Hora de apertura del bloque de atención.
hora_fin	Time	Hora de cierre del bloque de atención.
activa	Boolean	Desactivación lógica temporal del bloque horario.

3.4 Paciente

Registro de los usuarios receptores del servicio de salud. El modelo admite la parametrización de múltiples documentos de identidad bajo la legislación colombiana (Cédula de Ciudadanía - CC, Tarjeta de Identidad - TI, Cédula de Extranjería - CE, Pasaporte - PA).

3.5 Agendamiento (Entidad Central del Dominio)

Representa la reserva formal de una cita médica. Concentra las reglas más complejas del negocio y conecta con los módulos de triaje, auditoría y plataformas de comunicación.

Campo	Tipo de Datos	Restricciones / Descripción
codigo_cita	String(20)	Identificador alfanumérico único autogenerated con patrón: TM-XXXXXXXX (UK).
estado	Enum	Estados: PENDIENTE, CONFIRMADA, EN_CURSO, COMPLETADA,

Campo	Tipo de Datos	Restricciones / Descripción
		CANCELADA, NO_ASISTIO.
tipo_consulta	Enum	Clasificación: PRIMERA_VEZ, CONTROL, URGENCIA, SEGUIMIENTO.
modalidad	Enum	Canal de atención: VIDEOCONSULTA, CHAT, TELEFONO.
nivel_triaje	Enum	Escala Manchester: VERDE, AMARILLO, NARANJA, ROJO.
puntaje_triaje	Integer	Evaluación cuantitativa externa en un rango numérico de 0 a 100.
enlace_videoconsulta	String(500)	URL única apuntando a la sala remota de atención.
hora_fin	Time	Atributo calculado dinámicamente (hora_inicio + duracion_consulta_min).

Restricciones a Nivel de Base de Datos:

- Restricción de verificación (CHECK constraint) para garantizar que hora_fin sea cronológicamente posterior a hora_inicio.
- Índices compuestos multiactivos sobre (fecha_cita + medico_id) y (fecha_cita + paciente_id) diseñados para optimizar el rendimiento de las consultas e impedir solapamientos en la base de datos.

3.6 HistorialAgendamiento

Componente de auditoría y trazabilidad. Registra de forma determinista cada mutación de estado sufrida por una cita, almacenando el estado previo, el estado posterior, la estampa de tiempo exacta y el identificador del usuario responsable del cambio. Se invoca automáticamente desde el método cambiar_estado().

3.7 Usuario

Entidad núcleo del subsistema de autenticación y seguridad. Permite vinculación opcional uno a uno con registros de personal médico o pacientes. El control de acceso basado en roles se discrimina en: ADMIN, MEDICO y PACIENTE.

4. Autenticación y Autorización

El microservicio permite dos estrategias de inicio de sesión basadas en JSON Web Tokens (JWT):

4.1 Mecanismo por Encabezado (Header Bearer Token)

Orientado a la integración con aplicaciones móviles y consumo externo de APIs. El cliente realiza una petición de tipo POST hacia `/api/v1/auth/login` enviando las credenciales. Al recibir el token en el cuerpo de la respuesta JSON, está obligado a enviarlo en las solicitudes subsecuentes dentro de las cabeceras HTTP:

Authorization: Bearer <access_token>

4.2 Mecanismo por Cookies Seguras (Cookie HTTP-Only)

Estrategia recomendada para arquitecturas web de una sola página (SPA). El flujo se inicia mediante un POST al endpoint `/api/v1/auth/login_cookie`. El servidor responde inyectando la cookie `access_token_cookie` configurada con la bandera `HttpOnly` para mitigar vectores de ataque de scripts (XSS).

4.3 Matriz de Control de Acceso Basado en Roles (RBAC)

Rol del Sistema	Permisos y Alcance de Acceso
ADMIN	Acceso irrestricto al sistema. Facilidad de creación, modificación y borrado lógico de médicos, pacientes, especialidades y disponibilidad.
MEDICO	Permisos de escritura acotados al autoabastecimiento de su disponibilidad semanal. Permisos de lectura global sobre el agendamiento de citas asociadas a su registro.
PACIENTE	Capacidad de autogestión: consulta de su propio historial clínico y creación de nuevas citas. Restricción absoluta sobre datos pertenecientes a terceros pacientes.

Rol del Sistema	Permisos y Alcance de Acceso
Cualquier JWT Válido	Operaciones generales de lectura sobre catálogos de médicos y especialidades médicas.

5. Referencia de Endpoints

Todos los recursos se exponen bajo el prefijo común /api/v1. Las respuestas estructuradas por la API adoptan de manera consistente las llaves success, data y message.

5.1 Módulo de Autenticación (/api/v1/auth)

Método	Endpoint	Requiere Auth	Descripción Técnica
POST	/auth/login	No	Valida credenciales e implementa el retorno de un JWT estándar dentro de la carga JSON.
POST	/auth/login_cookie	No	Autentica al usuario e inyecta el token de seguridad en una cookie HTTP-Only.
POST	/auth/logout	No	Revoca la sesión eliminando las cookies del contexto del cliente.
POST	/auth/register	No	Endpoint público para el registro autónomo de nuevos usuarios con rol PACIENTE.
GET	/auth/me	Sí (JWT)	Recupera y retorna el perfil y metadatos del usuario firmante de la petición.

5.2 Módulo de Agendamientos (/api/v1/agendamientos)

Método	Endpoint	Requiere Auth	Descripción Técnica
GET	/agendamientos	Sí (JWT)	Recuperación de citas aplicando filtros de búsqueda (paciente_id, medico_id, estado, nivel_triage, modalidad, fechas). Paginado.
POST	/agendamientos	Sí (JWT)	Transacción para la reserva de citas. Ejecuta internamente el algoritmo de validación de disponibilidad y concurrencia horaria.
GET	/agendamientos/{id}	Sí (JWT)	Recuperación detallada de una cita médica referenciada por su clave primaria interna.
GET	/agendamientos/codigo/{cod}	Sí (JWT)	Resolución de la entidad de agendamento empleando el código de negocio estructurado TM-XXXXXXX.
PATCH	/agendamientos/{id}/estado	Sí (JWT)	Actualización parcial del estado. Evalúa la máquina de estados y exige un argumento de justificación si el destino es CANCELADA.

Método	Endpoint	Requiere Auth	Descripción Técnica
DELETE	/agendamientos/{id}	Sí (JWT)	Ejecución de cancelación lógica (soft cancel). Funciona como un alias operativo de PATCH estado=CANCELADA.
GET	/agendamientos/{id}/historial	Sí (JWT)	Consulta de la tabla de auditoría para la reconstrucción cronológica de los cambios sobre la cita.
GET	/agendamientos/slots	Sí (JWT)	Calcula la matriz de franjas horarias operativas detallando estados de ocupación (Parámetros: medico_id y fecha).

5.3 Módulo de Médicos (/api/v1/medicos)

Método	Endpoint	Rol Requerido	Descripción Técnica
GET	/medicos	JWT	Retorna el catálogo de personal médico en estado activo. Permite el filtrado por la llave especialidad_id.
POST	/medicos	JWT + ADMIN	Inserta un nuevo registro médico en el sistema, validando la unicidad del documento y el correo electrónico.

Método	Endpoint	Rol Requerido	Descripción Técnica
GET	/medicos/{id}	JWT	Provee la ficha técnica y de contacto de un médico en específico.
PUT	/medicos/{id}	JWT + ADMIN	Modificación integral de campos mutables (nombres, tarifa, duración de consulta).

5.4 Módulo de Pacientes (/api/v1/pacientes)

Método	Endpoint	Rol Requerido	Descripción Técnica
GET	/pacientes	JWT	Lista general de los registros de pacientes dados de alta en la plataforma. Incorpora paginación de datos.
POST	/pacientes	JWT + ADMIN	Alta y registro manual de pacientes dentro del sistema de información.
GET	/pacientes/{id}	JWT	Consulta detallada del perfil del paciente por ID interno.
GET	/pacientes/documento/{numero}	JWT	Endpoint de búsqueda rápida empleando el identificador legal del ciudadano.
PUT	/pacientes/{id}	JWT + ADMIN	Actualización del perfil clínico y de los canales de contacto provistos por el

Método	Endpoint	Rol Requerido	Descripción Técnica
			paciente.

5.5 Especialidades y Disponibilidad

Los endpoints del recurso /especialidades ejecutan operaciones estandarizadas de consulta y persistencia controladas mediante roles directos (lectura abierta bajo tokens JWT, escrituras restringidas al rol ADMIN).

Por su parte, el recurso /disponibilidad asiste el mantenimiento de los bloques semanales permitiendo la inyección de horarios a usuarios con roles MEDICO (restringido a su propia agenda) y ADMIN.

6. Lógica de Negocio Central

6.1 Algoritmo de Creación de un Agendamiento

Al invocar la rutina crear_agendamiento(), el motor del servicio ejecuta de manera síncrona el siguiente set de validaciones atómicas en orden estricto; el fallo de cualquiera aborta la transacción devolviendo el código de error HTTP adecuado:

1. **Existencia y Vigencia del Profesional:** Verifica que el medico_id corresponda a un registro existente y con el flag de actividad activo.
2. **Existencia y Vigencia del Usuario:** Valida que el paciente_id esté registrado y habilitado en el sistema.
3. **Consistencia Cronológica Temporal:** Evalúa que la variable fecha_cita no contenga un valor temporal situado en el pasado de la línea de tiempo del servidor.
4. **Cálculo de la Dimensión Temporal:** Ejecuta la proyección del término de la cita calculando: $\text{hora_fin} = \text{hora_inicio} + \text{duracion_consulta_min}$.
5. **Validación de la Oferta Horaria:** Cruza el día de la semana y las horas solicitadas contra la matriz relacional de DisponibilidadMedico.
6. **Resolución de Conflictos Médicos:** Examina que el profesional de salud no cuente con solapamientos de horario con ninguna otra cita en estado activo (CONFIRMADA o EN_CURSO) para la misma fecha.
7. **Resolución de Conflictos del Paciente:** Comprueba que el usuario solicitante no posea otra cita médica programada en el mismo rango horario de manera simultánea.
8. **Generación de Identificador de Negocio:** Produce el código alfanumérico aleatorio bajo la máscara TM-XXXXXXXX. En caso de colisión de unicidad, el algoritmo ejecuta un bucle de reintentos estructurados.
9. **Persistencia y Registro de Auditoría:** Consolida los datos en la base de datos relacional e inicializa el historial emitiendo el estado inicial PENDIENTE.

6.2 Máquina de Estados del Agendamiento

El flujo de estados para las citas se rige bajo reglas de transición deterministas. La violación del modelo conceptual genera una excepción controlada, retornando un código HTTP 422

(Unprocessable Entity) junto con el mapa detallado de transiciones válidas.

Estado Origen	Estados Destino Permitidos
PENDIENTE	CONFIRMADA, CANCELADA
CONFIRMADA	EN_CURSO, CANCELADA, NO_ASISTIO
EN_CURSO	COMPLETADA, CANCELADA
COMPLETADA	Ninguno (Estado Terminal)
CANCELADA	Ninguno (Estado Terminal)
NO_ASISTIO	Ninguno (Estado Terminal)

Nota Operativa: Toda solicitud de transición hacia el estado CANCELADA requiere de forma mandatoria la provisión de la variable motivo_cancelacion dentro del cuerpo de la petición.

6.3 Algoritmo de Cálculo de Slots Disponibles

El método asociado al endpoint GET /api/v1/agendamientos/slots procesa dinámicamente las ventanas temporales libres mediante el siguiente flujo computacional:

1. Determina el día de la semana basándose en el parámetro de entrada fecha.
2. Extrae los registros de la tabla DisponibilidadMedico que coincidan con el médico y estén vigentes para ese día específico.
3. Fragmenta los bloques horarios del médico en subintervalos de tiempo regulares basados en la variable duracion_consulta_min.
4. Ejecuta un barrido lógico confrontando cada franja contra los registros de la entidad Agendamento ya almacenados en la base de datos para esa fecha.
5. Asigna a cada slot el estado binario correspondiente (disponible: true o disponible: false) y compila contadores agregados de slots totales, ocupados y disponibles.

7. Indicadores Clave de Rendimiento (KPIs)

El monitoreo de la API se divide en dos perspectivas fundamentales: métricas funcionales orientadas al negocio de salud y métricas de rendimiento puramente técnicas de la infraestructura de software.

7.1 KPIs de Operación del Negocio

Indicador	Expresión Matemática / Fórmula de Cálculo	Umbral de Alerta / Meta
Tasa de Citas Completadas	$(\text{Citas COMPLETADA} / \text{Total de Citas}) * 100$	Meta: > 80%
Tasa de Cancelación	$(\text{Citas CANCELADA} / \text{Total de Citas}) * 100$	Alerta si: > 15%
Tasa de Inasistencia (No-Show)	$(\text{Citas NO_ASISTIO} / \text{Total de Citas}) * 100$	Alerta si: > 10%
Ocupación de la Agenda	$(\text{Slots Ocupados} / \text{Total de Slots Ofrecidos}) * 100$	Meta: > 70%
Tiempo de Confirmación	Tiempo de transición (PENDIENTE -> CONFIRMADA)	Meta: < 24 horas
Cobertura de Triage	$(\text{Citas con nivel_traje} / \text{Total de Citas}) * 100$	Análisis de tendencia
Volumen de Casos Críticos	Conteo absoluto de registros WHERE triaje IN (ROJO, NARANJA)	Monitoreo continuo síncrono

7.2 KPIs Técnicos de la API

Indicador	Objeto de Medición	Métrica Objetivo (SLA)
Disponibilidad (Uptime)	Tiempo en línea mensual del microservicio.	Target: > 99.5%
Latencia P95	Tiempo de respuesta del percentil 95 en peticiones HTTP.	Target: < 300 ms
Tasa de Errores 5xx	Proporción de errores internos del servidor frente al total de solicitudes.	Límite tolerado: < 1%

Indicador	Objeto de Medición	Métrica Objetivo (SLA)
Tasa de Éxito de Autenticación	Ratio de llamadas exitosas al endpoint de login contra intentos totales.	Target: > 90%
Saturación por Concurrencia	Volumen de respuestas HTTP 409 (Conflict) sobre el endpoint POST.	Límite tolerado: < 5%
Rendimiento de Slots	Latencia específica del procesamiento algorítmico en /slots.	Target: < 500 ms
Rechazos por Token	Ratio de respuestas HTTP 401 debido a la expiración de firmas JWT.	Límite tolerado: < 2%
Cobertura de Auditoría	Integridad de los eventos reflejados en la tabla de historial.	Meta: 100% obligatoria

8. Configuración y Despliegue

8.1 Variables de Entorno del Sistema

Variable de Entorno	Tipo / Estado	Propósito y Aplicación en el Sistema
DATABASE_URL	String / Requerido	Cadena de conexión al motor PostgreSQL (Mandatorio en entornos productivos).
SECRET_KEY	String / Crítico	Clave secreta criptográfica utilizada para asegurar las sesiones nativas de Flask.
JWT_SECRET_KEY	String / Crítico	Semilla criptográfica empleada para la firma simétrica de los tokens JWT.

Variable de Entorno	Tipo / Estado	Propósito y Aplicación en el Sistema
JWT_ACCESS_TOKEN_EXPIRES	Integer / Opcional	Tiempo de vigencia del token medido en segundos (Valor por defecto: 3600).
JWT_TOKEN_LOCATION	String / Opcional	Configura los puntos de extracción del token: headers o cookies.
JWT_COOKIE_SECURE	Boolean / Requerido	Restringe la transmisión del token únicamente sobre canales HTTPS cuando se asigna en True.
CORS_ORIGINS	String / Requerido	Orígenes explícitos permitidos por el servidor para el control de acceso en navegadores web.

8.2 Despliegue Mediante Contenedores (Docker)

El microservicio se encuentra completamente contenerizado. La ejecución del comando: `docker compose up --build`

Desencadena la siguiente cadena de eventos: la compilación de la imagen basada en un entorno optimizado de Python, la instalación de las dependencias especificadas en el manifiesto, la ejecución automática de las migraciones de base de datos (`flask db upgrade`) y el levantamiento de la aplicación junto a una instancia aislada de PostgreSQL configurada en el archivo `docker-compose.yml`.

8.3 Gestión de Migraciones de Base de Datos

El control de cambios sobre los esquemas de datos relacionales es gestionado de manera estricta por Alembic a través de los wrappers de Flask.

- Para aplicar cambios pendientes al motor de base de datos activo: `flask db upgrade`
- Para generar un nuevo script de migración tras modificar un archivo de modelos: `flask db migrate -m "descripcion"`

9. Integración con el Módulo de Triage

El microservicio cuenta con un acoplamiento débil con el software de triaje automático, permitiendo una integración asíncrona y segura a través de los atributos expuestos en el modelo Agendamiento:

- **Atributo nivel_triaje:** Recibe los estados de clasificación basados en el protocolo internacional Manchester (VERDE, AMARILLO, NARANJA, ROJO).
- **Atributo puntaje_triaje:** Almacena la ponderación fina generada de forma externa mediante algoritmos de inteligencia artificial, mapeada en un formato de entero entre 0 y 100.

Los sistemas externos de triaje pueden interactuar con las citas mediante el consumo de los verbos POST y PATCH para adjuntar la clasificación diagnóstica inicial. El microservicio expone facilidades de filtrado en el endpoint de consulta masiva (GET /api/v1/agendamientos?nivel_triaje=ROJO), permitiendo a las interfaces médicas priorizar de forma automática las citas de pacientes en estado crítico.

10. Notas Técnicas y Recomendaciones de Mejora

A partir del análisis del estado actual del repositorio de código, se han detectado brechas técnicas que se recomienda mitigar en los próximos ciclos de trabajo continuo bajo el marco Kanban:

1. **Implementación de Rate Limiting:** La API carece de restricciones frente a ráfagas masivas de solicitudes. Se sugiere integrar de forma prioritaria la librería flask-limiter, enfocando las políticas de restricción de tráfico en el endpoint crítico de autenticación /auth/login con el fin de neutralizar ataques de fuerza bruta.
2. **Desarrollo del Subsistema de Notificaciones:** El modelo expone el campo booleano recordatorio_enviado; sin embargo, no existe un motor de servicios asociado. Se propone construir un microservicio de mensajería (SMS/Email) conectado mediante eventos para automatizar el despacho de alertas al paciente.
3. **Automatización de Salas de Videoconferencia:** La columna enlace_videoconsulta funciona exclusivamente como un campo contenedor de cadenas de texto de forma estática. Se recomienda incorporar integraciones basadas en Webhooks o SDKs de proveedores externos especializados (como Daily.co, Whereby o Jitsi) para dinamizar la creación de salas en el cambio de estado hacia CONFIRMADA.
4. **Estandarización de la Documentación OpenAPI:** El proyecto no expone de manera nativa un archivo de especificación técnica descriptivo (swagger.yaml). Es imperativo añadir utilidades de generación dinámica como flask-smorest o flasgger para agilizar la integración con los equipos de desarrollo Frontend.
5. **Ampliación de la Cobertura de Pruebas Unitarias:** Si bien el proyecto posee un directorio base de pruebas (tests/), la cobertura es baja. Es crítico diseñar un set exhaustivo de pruebas unitarias enfocado en la validación de las aristas del algoritmo de búsqueda de conflictos horarios y la robustez de las transiciones de la máquina de estados.